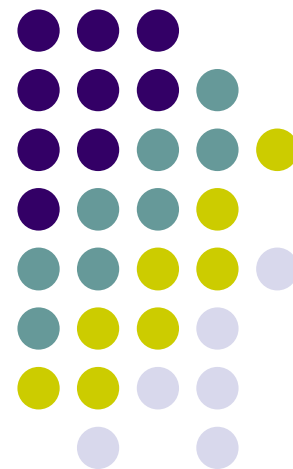
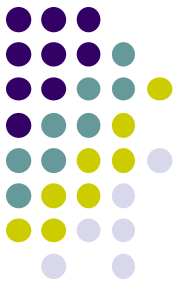


L24: BST和AVL

吉林大学计算机学院
谷方明

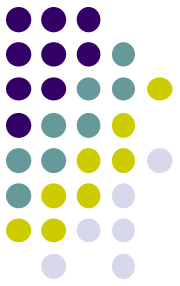
fmgu2002@sina.com





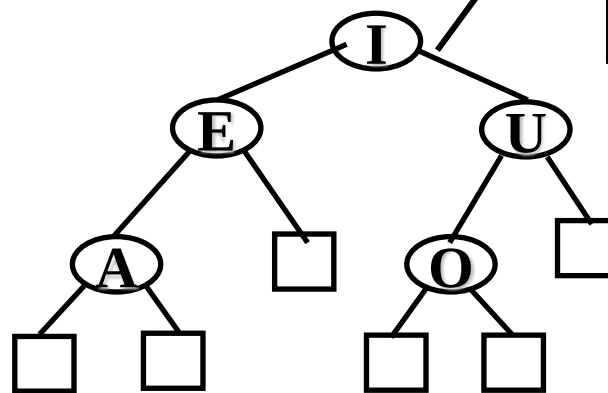
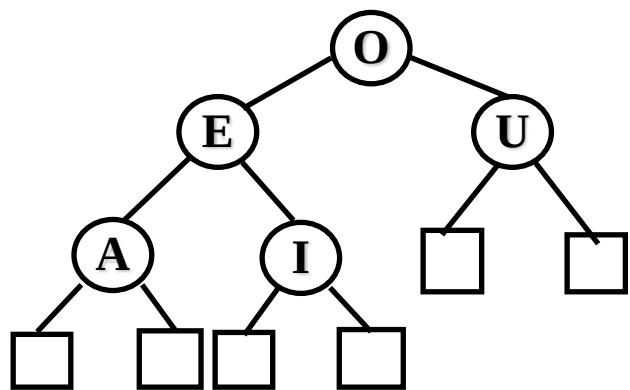
查找树的引入

- 对半查找、斐波那契查找等都对应一棵**隐式的(implicit)**二叉树；
- 想法：用一个**显式的(explicit)**二叉树结构组织数据进行查找；
 - ✓ 不但能对表进行有效查找，
 - ✓ 还能迅速插入和删除（动态表）。

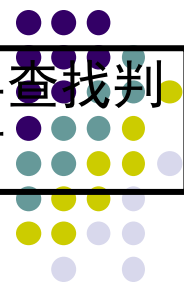


二叉查找树(BST)概念

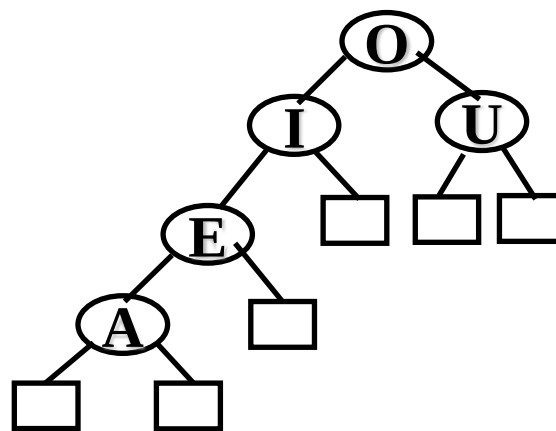
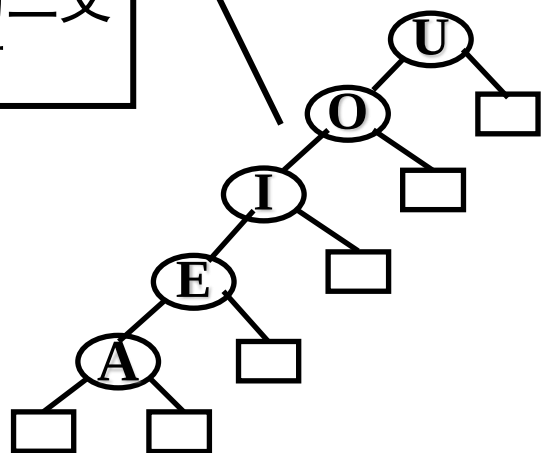
- **BST左子树中结点的关键词都小于 $KEY(root)$ ，其右子树中结点的关键词都大于 $KEY(root)$ ，且左右子树都是二叉查找树**
- **递归版定义，可编写递归算法**

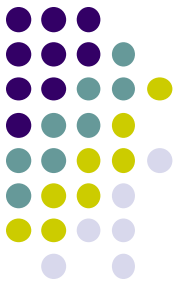


对半查找判定树



退化的二叉查找树





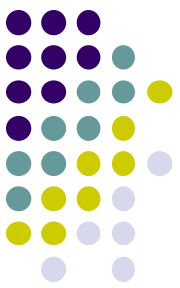
二叉查找树VS对半查找判定树

□ 联系

- ✓ 对半查找判定树是一种二叉查找树
- ✓ 都可用于描述查找

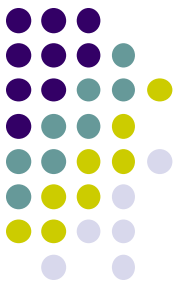
□ 区别

- ✓ 二叉查找树是一种显式树结构；对半查找判定树是一种隐式树结构；
- ✓ 二叉查找树直接用于查找；对半查找判定树用于分析查找



BST其它定义

- BST中任一结点 P ，其左子树中结点的关键词都小于 $KEY(P)$ ，其右子树中结点的关键词都大于 $KEY(P)$ 。
 - ✓ 二叉搜索树性质，可迭代处理。
- 定义8.1 二叉查找树（二叉搜索树、二叉排序树）：
BST是一棵可为空的二叉树；若BST非空，则其中根遍历序列按关键词递增排列。
 - ✓ 中序遍历有序，故又称二叉排序树



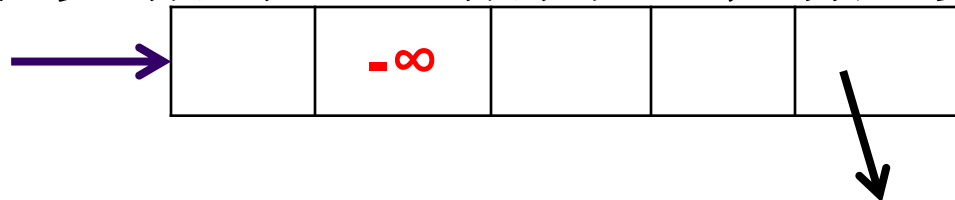
二叉查找树的存储结构

- 链式存储：结点结构如下

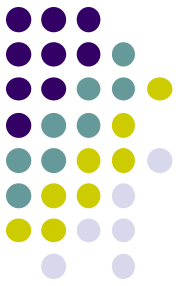
left	key	data	parent	right
------	-----	------	--------	-------

LEFT和**RIGHT**是链接字段，**KEY**为关键词。

- 可让表头指针**root**指向一个哨兵变量，最小

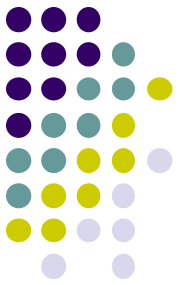


left	key	data	parent	right
------	-----	------	--------	-------



二叉查找树的基本操作

- 查找 (key-value)
- FindMin/FindMax
- Predecessor/Successor
- 插入
- 删除
-



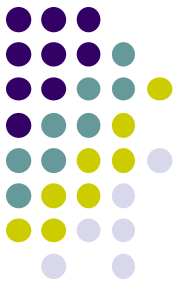
BST的操作分析

□ BST既可以作优先级队列

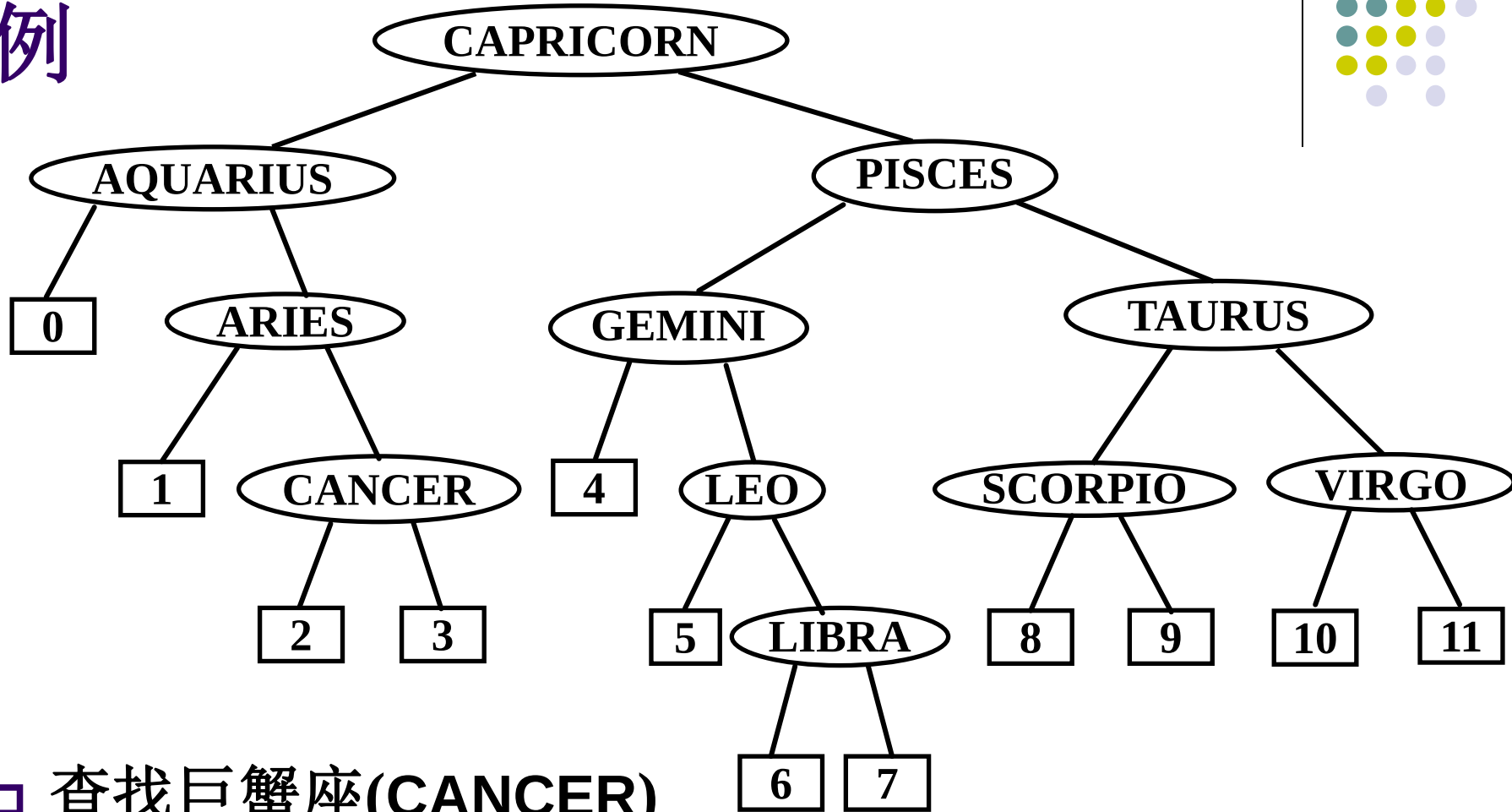
- ✓ 优先级队列中的元素具有**优先级**
- ✓ 优先级队列的操作：取最值、删最值、插入、修改

□ BST也可以作字典。

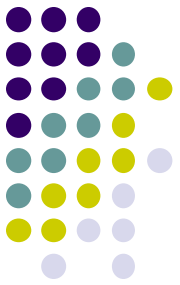
- ✓ 字典是key-value对的集合。也称为关联数组（associative array）或映射（**map**）
- ✓ 字典的操作：查找、插入、删除。



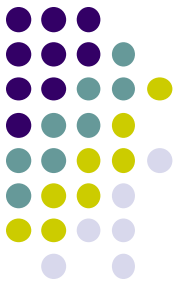
例



- ❑ 查找巨蟹座(CANCER)
- ❑ 查找第12个名字人马座 (SAGITTARIUS)



- 设表中元素的关键词 $K_1 < K_2 < \dots < K_N$ ，则查找成功应该终止于 R_i (内结点)，而查找失败应终止在 $N+1$ 个记录间隔(或者称外结点，即 $K_i < K < K_{N+1}$ 的情形)。
- 查找失败可以直接返回，也可以在失败位置插入新结点，称为查找与插入操作；



Find (t, k)

算法 Find (t, k)

/*在t为根的树中查找关键词为k的结点，迭代版*/

F1 [由根迭代比较]

c = t;

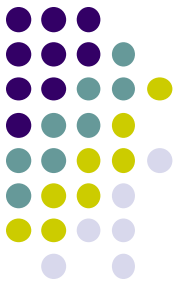
while (c != NULL && c->key != k)

if (c < c->key) c=c->left; else c=c->right;

return c;

//时间复杂度： 树高 $O(h)$

//递归版自己实现



算法FindWithParent (t, k.p,pp)

*/*在t为根的树中查找关键词为k的结点及其双亲*/*

FP1 [由根迭代比较]

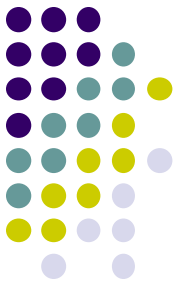
c = t, p=NULL;

while (c != NULL && c->key != k)

if (c < c->key) p=c, c=c->left;

else p=c, c=c->right;





FindMin/FindMax

□ 算法FindMin(t)

/* 在以t为根的BST中找最小元素*/

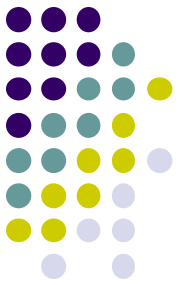
F1[一路向左]

if(t != NULL)

while(t->left != NULL) t = t->left;

return t;

//时间复杂度： 树高 $O(h)$



Predecessor/Successor

□ 算法Successor(x)

*/*在BST中找x的后继*/*

S1[有右儿子，向下找]

if(x.right != NULL) return FindMin(x.right);

S2[否则向上找，找到最近的祖先并在其左子树中]

y = x.parent;

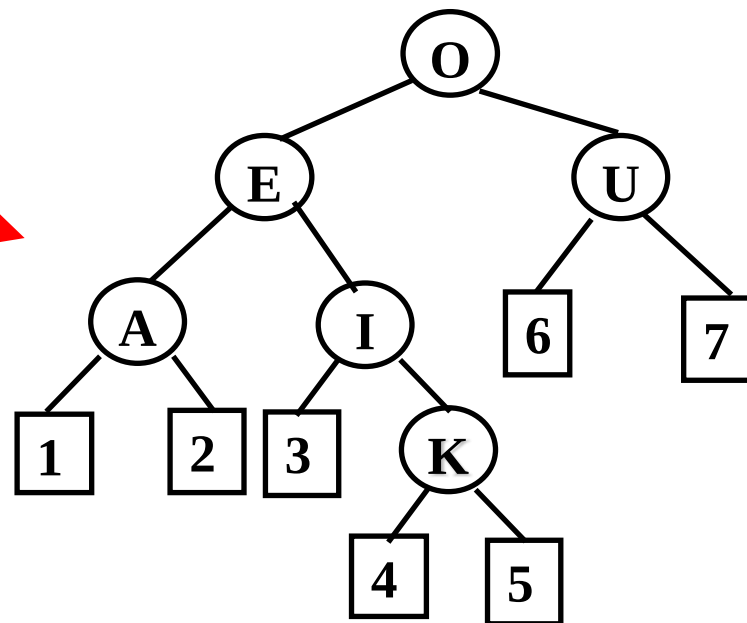
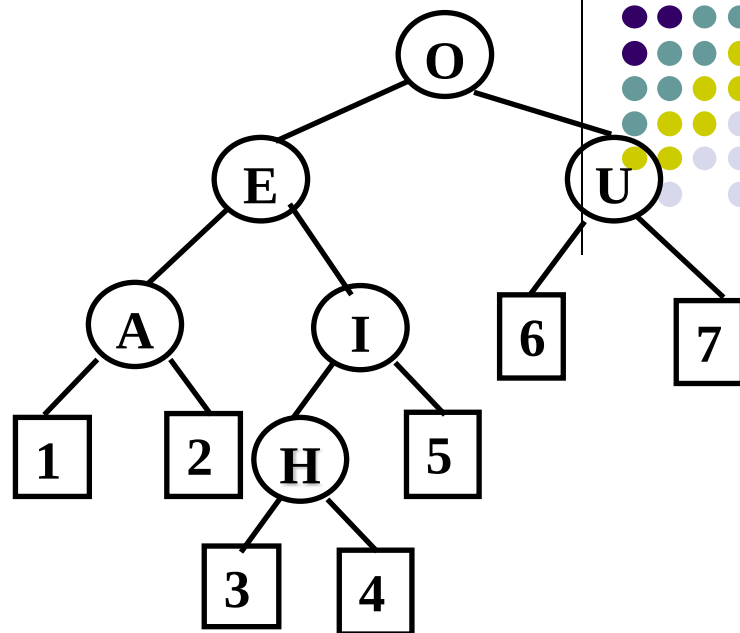
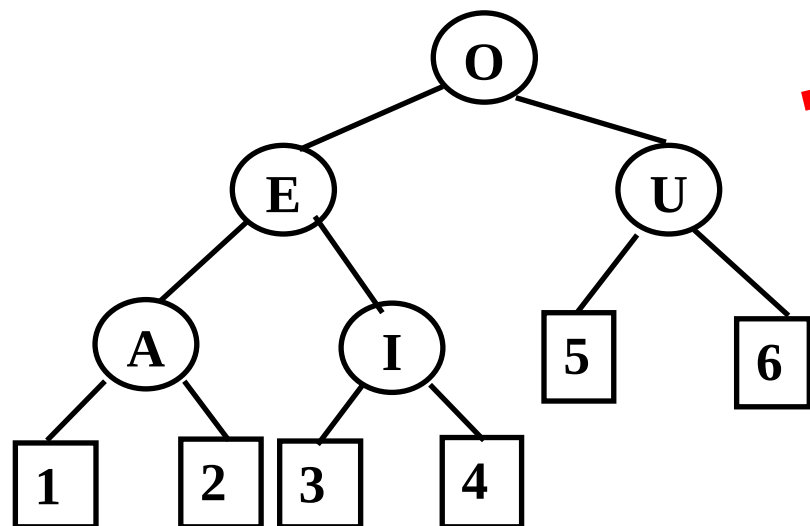
while(y!= NULL && x==y.right)

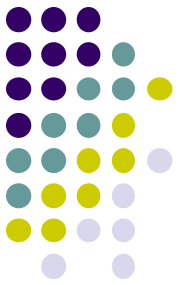
x=y, y=x.parent;

return y;

//时间复杂度： 向下或向上， $O(h)$

插入





插入操作（迭代版）

算法 Insert(t, k)

*/*在t为根的子树中寻找关键词为k的结点,失败时插入*/*

I1 [查找失败时插入]

FindWithParent(t,k,c,p);

if(c==NULL){

c $\leftarrow$ AVAIL ;

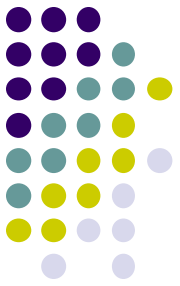
c->key = k; c->left = c->right = NULL;

c->parent = p; //父子链接，可封装函数

if(p)

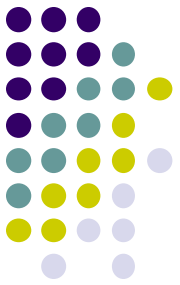
if(p->left==p) p->left = p else p->right = p;

}



创建

- 调用 n 次 Insert
- 从空二叉树开始，依次插入包含关键词 **CAPRICORN, AQUARIUS, PISCES, ARIES, TAURUS, GEMINI, CANCER, LEO, VIRGO, LIBRA**和**SCORPIO**之结点所形成的二叉查找树.
- 练习：输入序列为{45, 24, 53, 12, 37, 93}

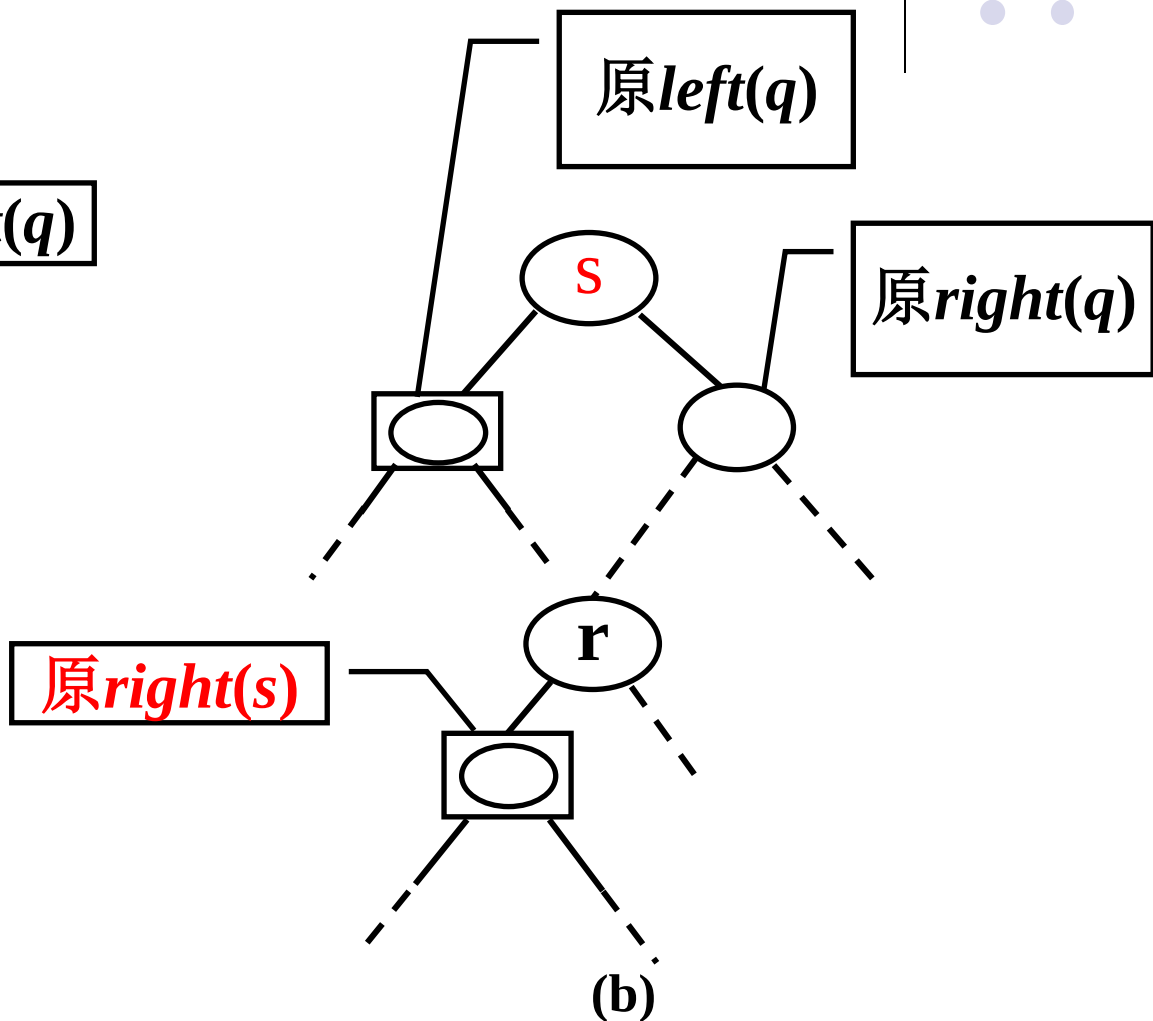
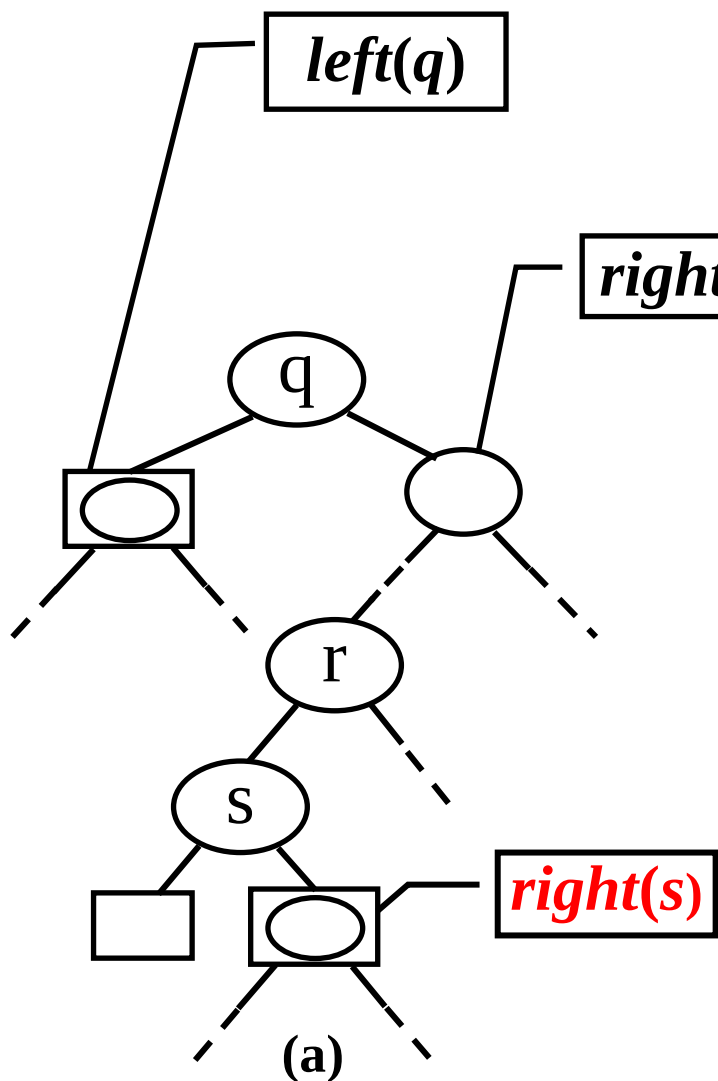
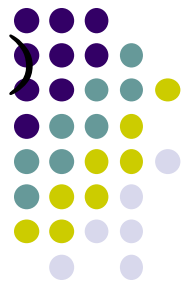


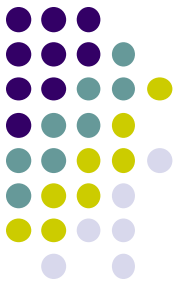
删除

- 假定指针 q 指向**BST**中待删除的结点；要求：删除**BST**后的树仍为二叉查找树。

- 删除分三种情况（实现时，1和2可合并）
 1. q 没有儿子；（双亲的对应域用空替换）
 2. q 只有一个儿子；（双亲的对应域用该儿子替换）
 3. q 有两个儿子：为保持中序递增，使用中根后继（或前驱）替换；

删除的第三种情况：使用中根后继替换（没真删q）





删除操作（迭代版）

算法 Delete(t, x) /*在根为t的树中删除结点x*/

D1 [确定实际删除位置d及其接替s]

d=x, s = NULL;

if(x->left == NULL) s = x->right;

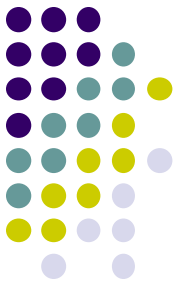
else if(x->right == NULL) s = x->left;

else{

d = Successor(x); swap(x->data, d->data);

s = d->right;

}



D2 [调整链接和删除]

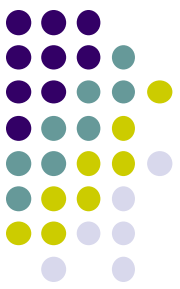
p = d->parent;

if(s!=NULL) s->parent = p;

if(p!=NULL)

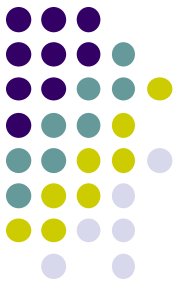
if(p->left == d) p->left = s; else p->right = s;

delete(d); //释放d



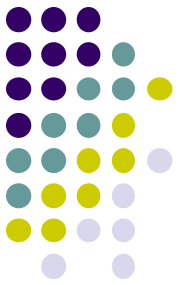
二叉查找树的简单分析

- 一个无序序列，可通过构造一棵二叉查找树而成为有序序列，这被称为二叉查找树的**顺序属性**。
- 二叉查找树会因输入文件的记录顺序不同，而有不同的形态。对包含 N 个记录的集合，其对应的关键词有 $N!$ 种不同的排列，可构成 $C_{2N}^N / (N + 1)$ 棵不同的二叉查找树。
- **特殊情况下，会产生退化二叉查找树，从而使最坏情况查找时间达 $O(N)$ 。**



BST的期望高度

- 当一棵二叉搜索树同时由插入和删除操作生成时，**很难精确衡量**这棵树的平均高度。
- 当树是由插入操作单独生成时，分析就容易得多
- **定理：一棵由n个不同关键字的随机构建的二叉搜索树的期望高度为 $O(\log n)$**
 - ✓ n个关键字的一棵随机构建的二叉搜索树为按随机次序插入这些关键字到棵初始的空树中而生成的树，



证明I（教材）

- 找到一个关键词所需的比较次数，恰比 将此关键词插入树中所需的比较次数多**1** .

$$S_N = 1 + (U_0 + U_1 + \cdots + U_{N-1})/N$$

- 由引理**8.1**

$$S_N = (1+1/N) U_N - 1$$

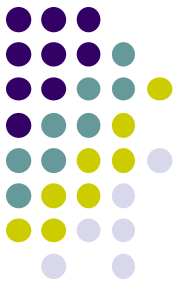
- 综合两公式，可得

$$(N + 1)U_N = 2N + (U_0 + U_1 + \cdots + U_{N-1})$$

$$N U_{N-1} = 2N - 2 + (U_0 + U_1 + \cdots + U_{N-2})$$

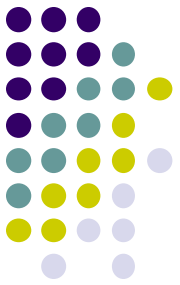
$$U_N = U_{N-1} + 2/(N + 1)$$

$$U_N = 2H(N + 1) - 2$$



证明II （算法导论）

- X_n : 一棵具有 n 个结点的随机构建BST的高度
- $Y_n = 2^{X_n}$: 指数高度
- Z_i : 指示器变量; i 为根结点, $Z_i = 1$, 否则0 ;
所有 Z_i 中, 恰有一个为1, 其余为0; $\Pr(Z_i = 1) = 1/n$ ($1 \leq i \leq n$)
- 假定第 i 个结点为根, 则
 - ✓ $X_n = 1 + \max\{X_{i-1}, X_{n-i}\}$
 - ✓ $Y_n = 2 \max(Y_{i-1}, Y_{n-i})$
- $Y_n = \sum Z_i (2 \max(Y_{i-1}, Y_{n-i}))$

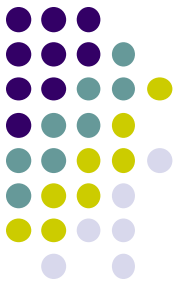


$$\begin{aligned}\square E(Y_n) &= E\left(\sum_{i=1}^n Z_i(2 \max(Y_{i-1}, Y_{n-i}))\right) \\ &= \sum_{i=1}^n \frac{2}{n} \max(E(Y_{i-1}), E(Y_{n-i})) \\ &\leq \sum_{i=1}^n \frac{2}{n} (E(Y_{i-1}) + E(Y_{n-i})) \\ &= \frac{4}{n} \sum_{i=0}^{n-1} E(Y_i)\end{aligned}$$

$$\square \text{ 往证: } E(Y_n) \leq \frac{1}{4} \binom{n+3}{3}$$

$\square$ 基础: $n=0$ 和 1 时, $Y_0=0$, $Y_1=1$, 均成立

$$\begin{aligned}\square \text{ 归纳: } E(Y_n) &\leq \frac{4}{n} \sum_{i=0}^{n-1} E(Y_i) \leq \frac{4}{n} \sum_{i=0}^{n-1} \frac{1}{4} \binom{i+3}{3} \\ &= \frac{1}{n} \sum_{i=0}^{n-1} \binom{i+3}{3} = \frac{1}{n} \binom{n+3}{4} = \frac{1}{4} \binom{n+3}{3}\end{aligned}$$



□ Jensen不等式:

对任意点集 $\{x_i\}$, 若 $\lambda_i \geq 0$ 且 $\sum \lambda_i = 1$, f 是凸函数,

$$f(\sum \lambda_i \cdot x_i) \leq \sum \lambda_i \cdot f(x_i)$$

□ 对期望和凸函数应用Jensen不等式:

$$f(E(X)) = f(\sum p_i \cdot x_i) \leq \sum p_i \cdot f(x_i) = E(f(X))$$

□ 因此:

$$2^{E(Xn)} \leq E(2^{Xn}) = E(Yn) \leq \frac{1}{4} \binom{n+3}{3}$$

$$E(Xn) = O(\log n)$$

□ 定理得证。

二叉搜索树的退化的解决方案 (开放问题)



□ 方案一：随机文件

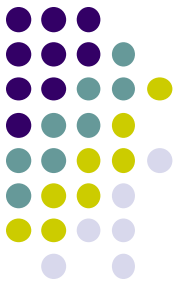
□ 方案二：最优二叉查找树（教材8.4节）

- ✓ 给定各种查找情况发生的概率 α_i 和 β_i ，建立一棵最优(最小加权通路长度)的二叉查找树；对树动态调整，保证插入删除结点后，仍为最优。
- ✓ 计算复杂度高，有时结点出现的频率无法精确统计

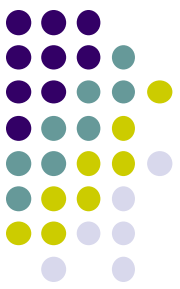
□ 方案三：AVL等平衡树

- ✓ 思想：控制树形维护一棵形态较好的二叉查找树（近似最优）

AVL树



- 1962年，两名俄国数学家G. M. **Adelson-Velsky**和E. M. **Landis**，发现了一个非常漂亮的维持一棵好的查找树的方案——**AVL** 树。仅要求为每个结点增加两个额外的二进制位。



高度平衡树

- **定义8.2**: 一棵二叉查找树称为高度平衡二叉查找树（高度平衡树、平衡树），当且仅当由单一的外结点组成，或者由两个子树形 T_l 和 T_r 组成，且满足：
 - (1) $|h(T_l) - h(T_r)| \leq 1$ ，其中 $h(T)$ 表示树 T 的高度；
 - (2) T_l 和 T_r 都是高度平衡树.
- **定义8.3**：设 T 为高度平衡树， q 是 T 任一内结点， q 的平衡系数（平衡因子） $BF(q)$ 定义为 $h_R - h_L$. 其中， h_L 和 h_R 分别 q 的左、右子树的高度。
- 显然 $BF(q) = 0, 1$ 或 -1

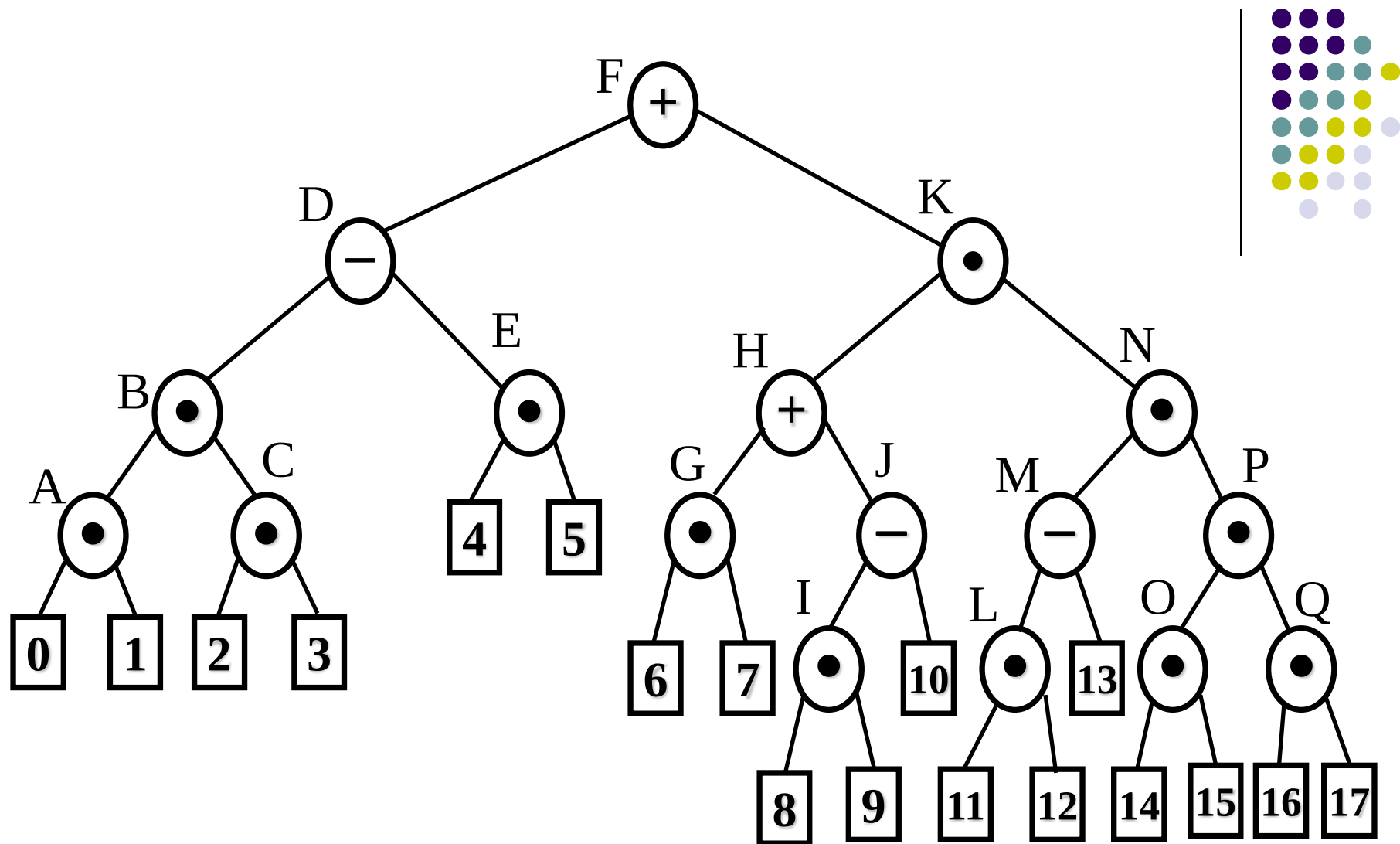
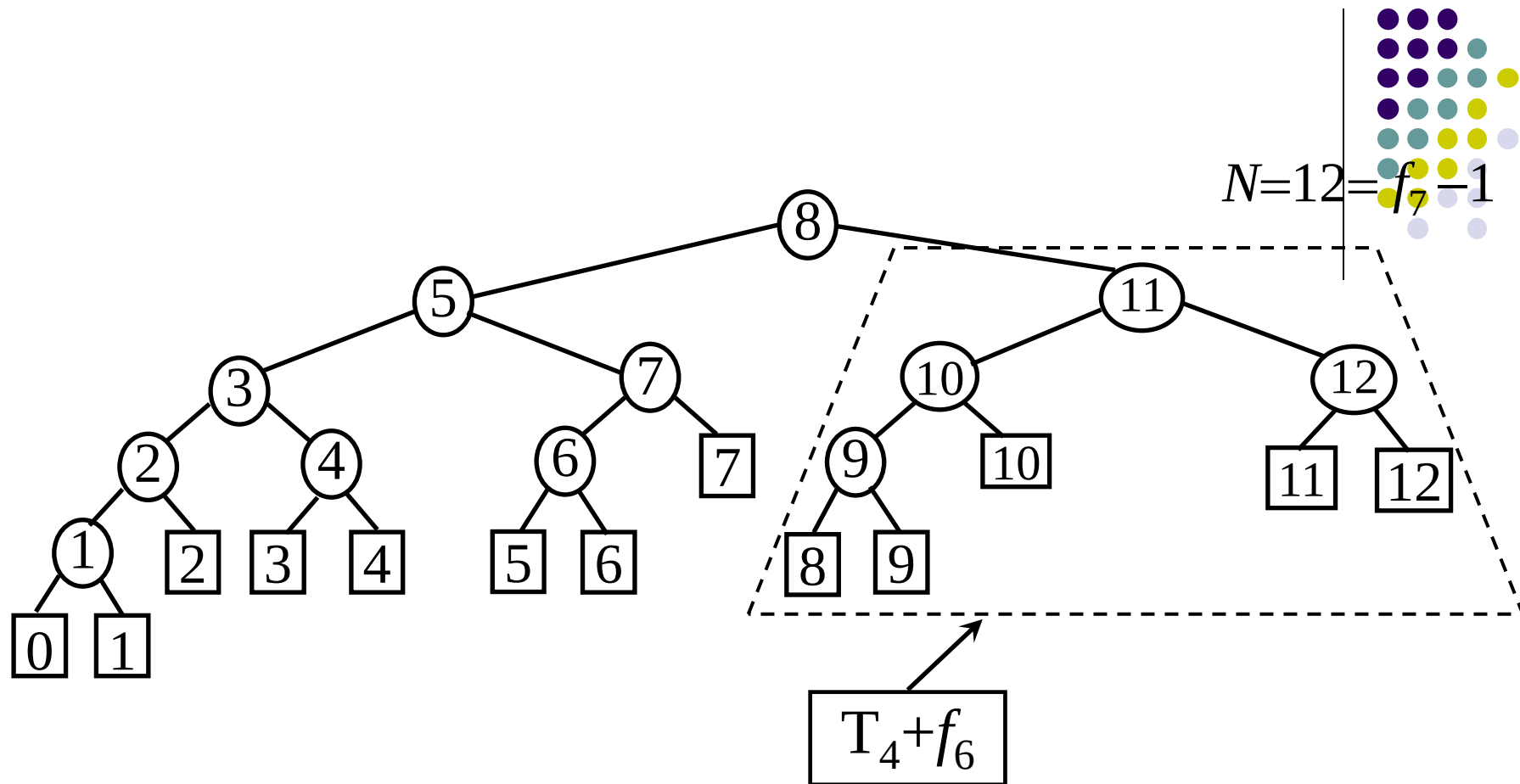
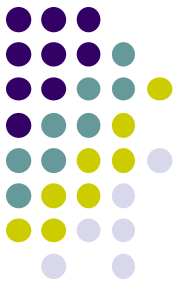


图8.23 一棵平衡二叉树形



- 斐波那契判定树是一棵高度平衡树，且每个结点 (以该结点为根的二叉树的内结点数 ≥ 2) 的平衡系数为 -1 ，该判定树在内结点数相同的高度平衡树中具有最大高度。



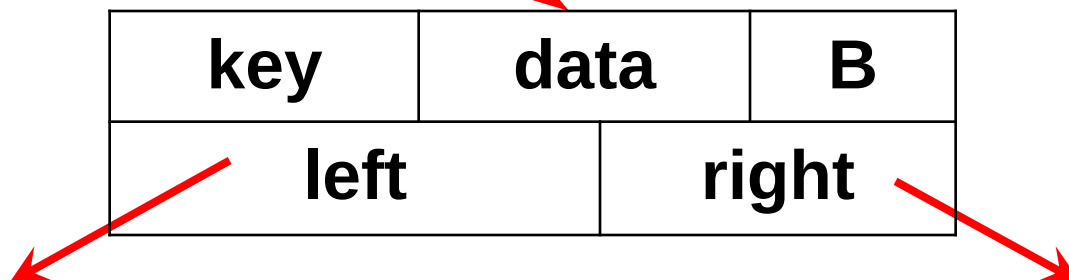
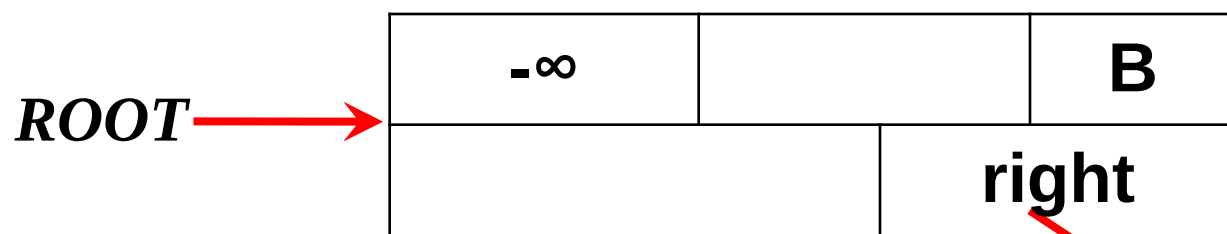
AVL的高度

- 定理8.4 (Adelson-Velsky和Landis) 设 T 是一棵具有 n 个内结点的平衡树, T 之高度 $h(T)$ 由下式所限定
$$\log_2(n+1) \leq h(T) \leq 1.4404 \times \log_2(n+2) - 0.3277$$
- 平衡树的查找路径长度决不会超过最优树的查找路径长度的45% .

存储结构

- 结点结构
- 哨位结点

key	data	B
left		right



查找操作

□ 同BST

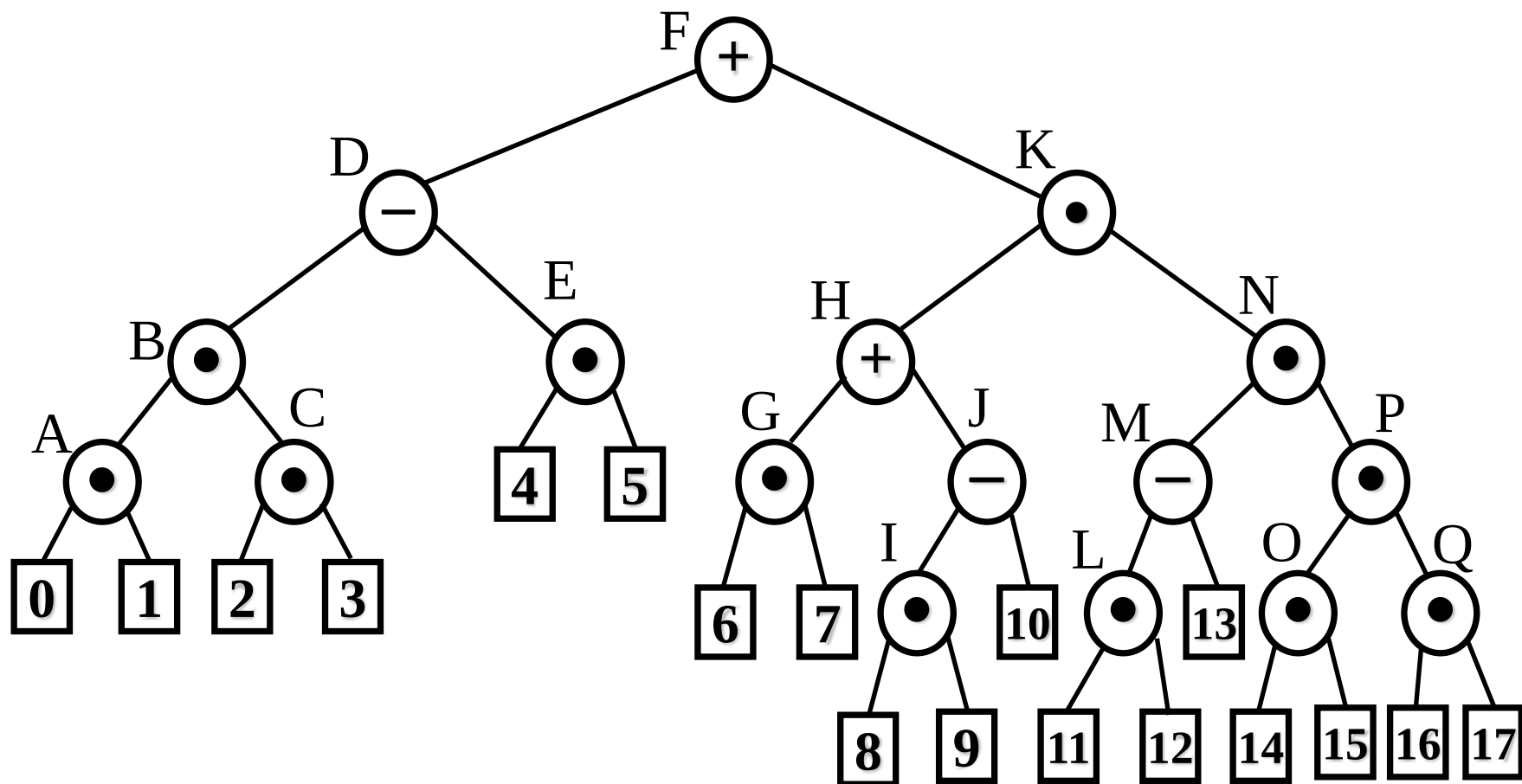


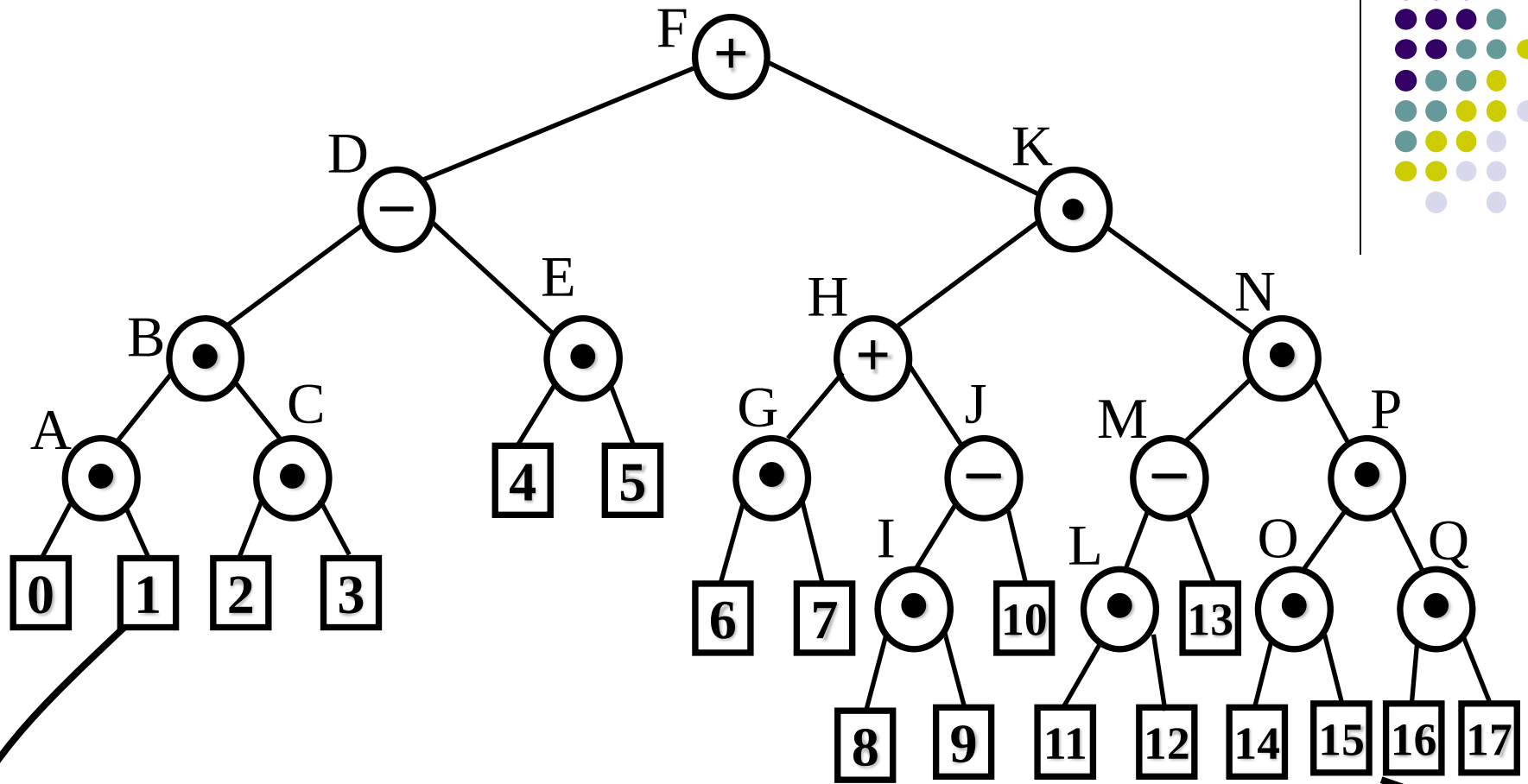
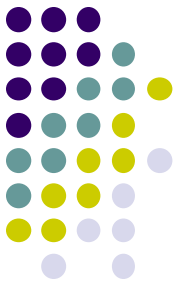
插入



在高度平衡树上插入结点，可能会破坏平衡性。在

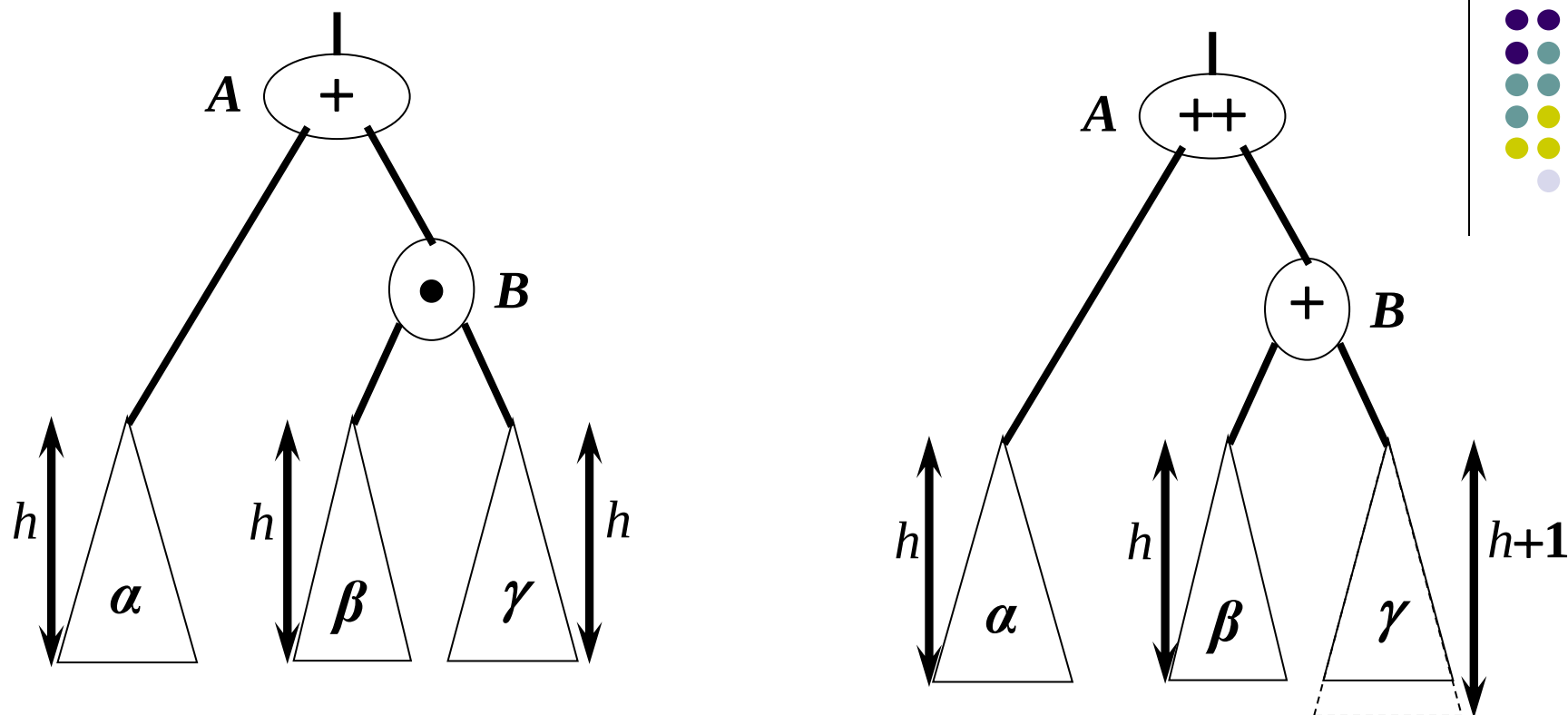
4，**5**，**6**，**7**，**10**，**13** 等处插入无损平衡性。





□ 平衡性遭到破坏的情况:

- ✓ 平衡系数为+1的结点, 如果在它的右子树的外结点上插入新结点, 使它的右子树变得更高
- ✓ 平衡系数为-1的结点, 如果在它的左子树的叶结点上插入新结点, 使它的左子树变得更高

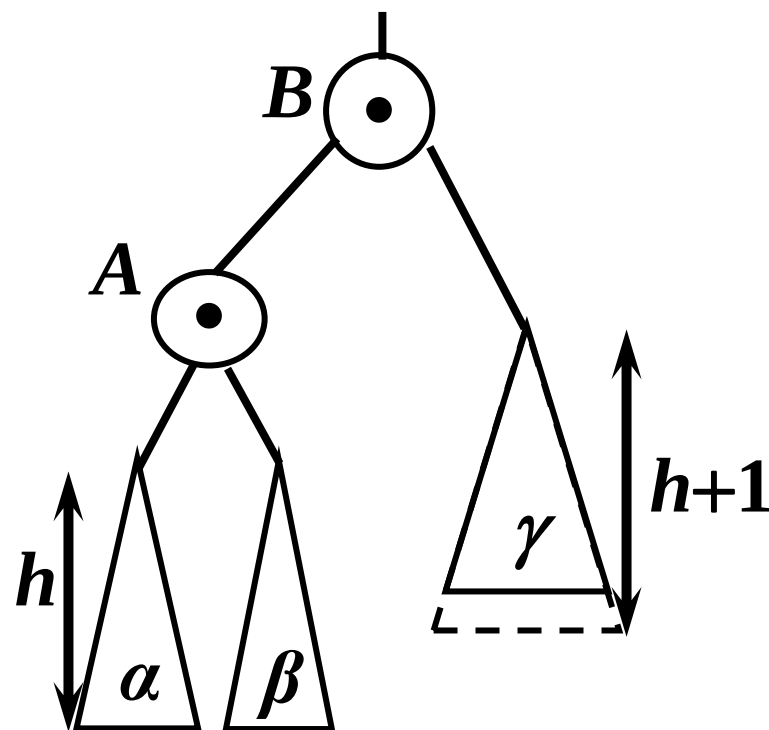
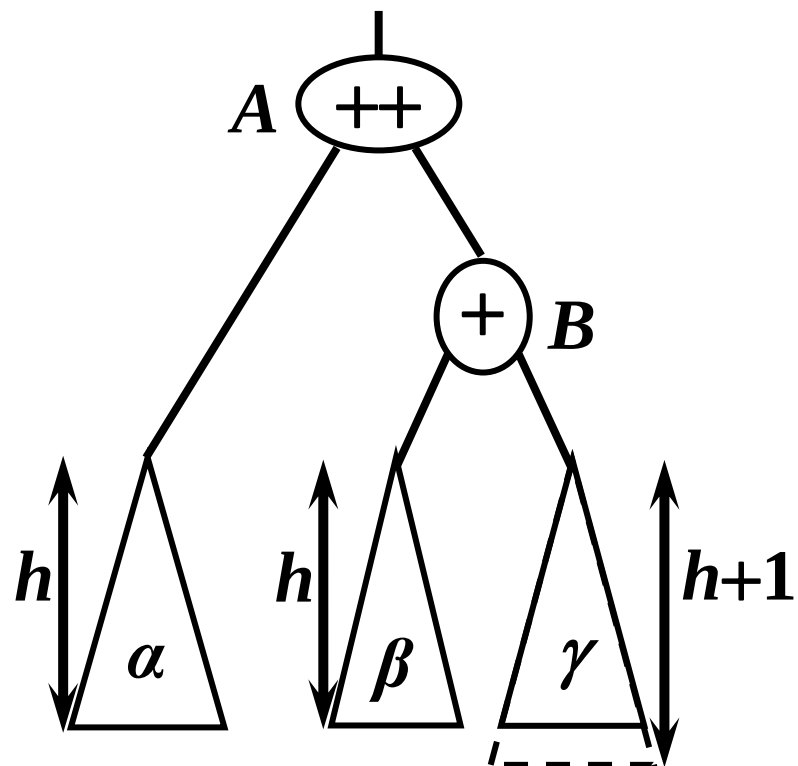


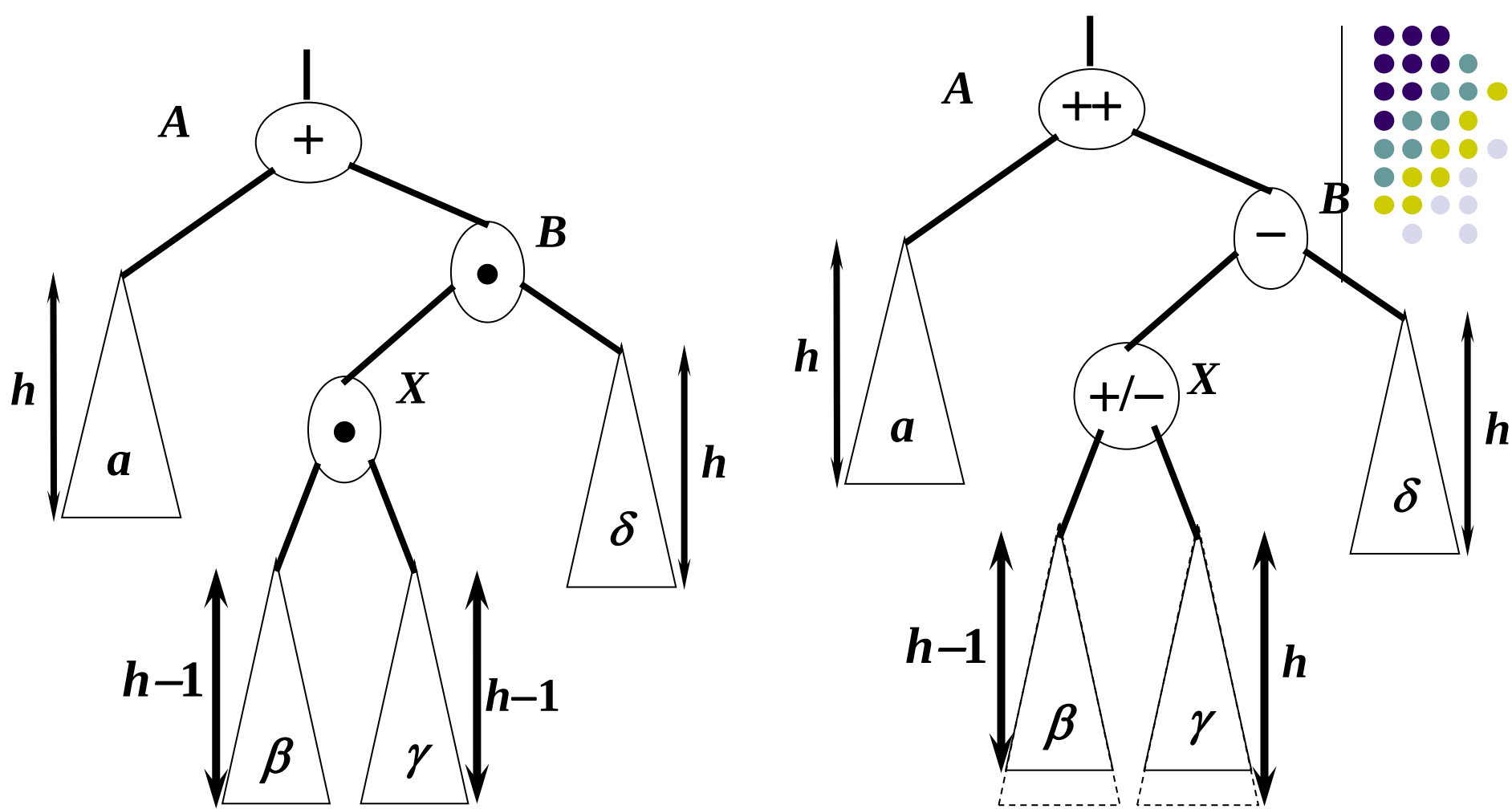
(a) 情况1

图8.24 破坏平衡的情况1（虚框表示新插入结点的位置）
（若反演改图式，左右交换，则会出现另外1种相同的情况）

情况1: “单一转动”。

把结点 B 从 A 的右下侧左转到 A 的右上侧, 原 B 之左子树 β 变成了 A 的右子树, B 的新左儿子是结点 A 。





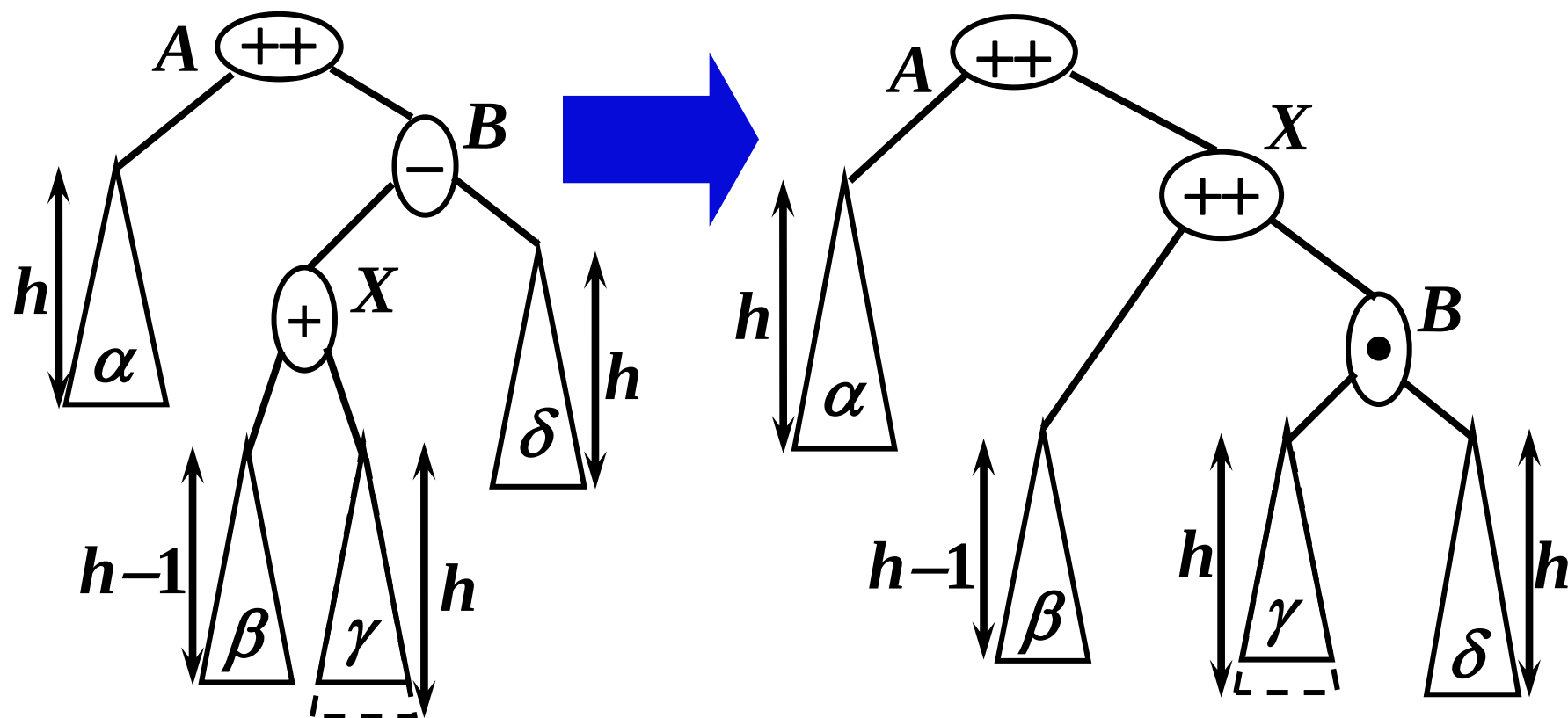
(a) 情况2

图8.24 破坏平衡的情况2（虚框表示新插入结点的位置）
（若反演该图式，左右交换，则会出现另外1种相同的情况）

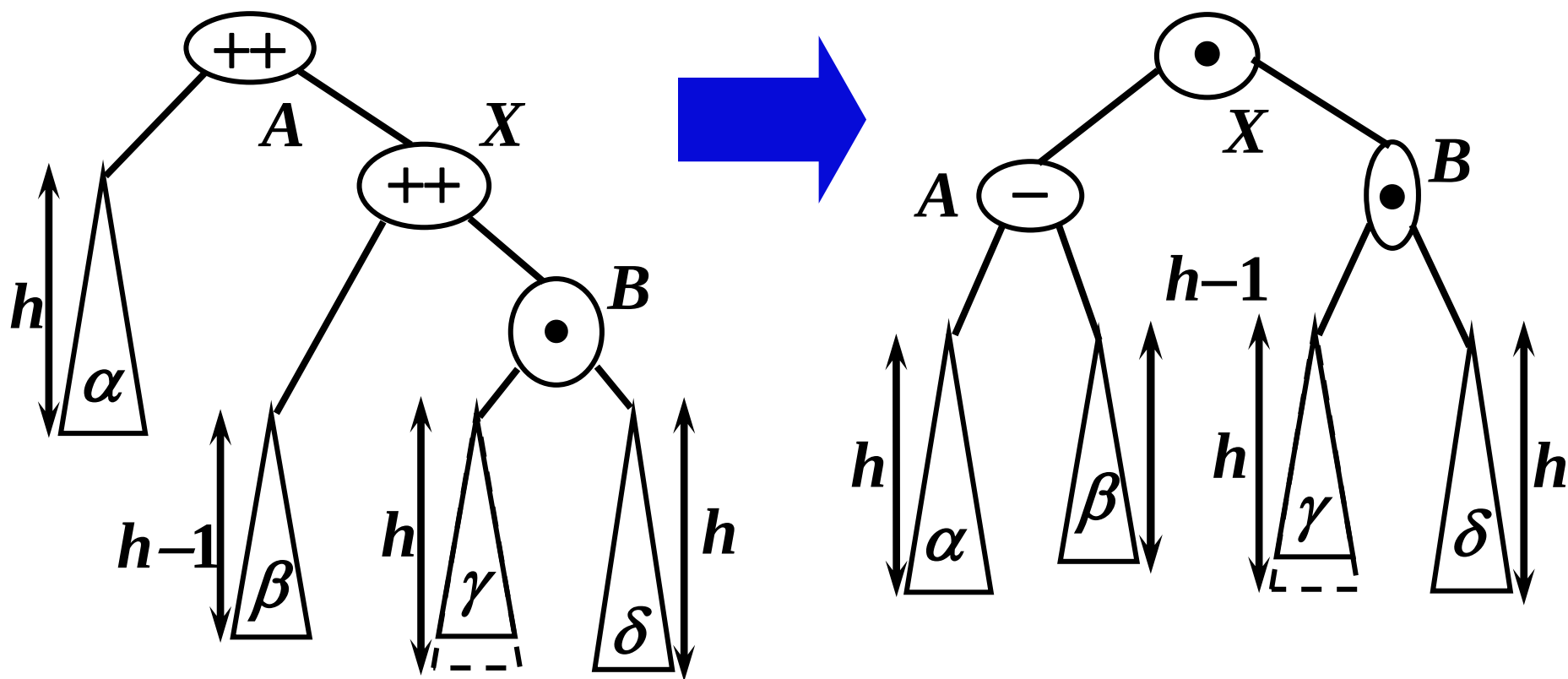


情况2: “双重转动”。

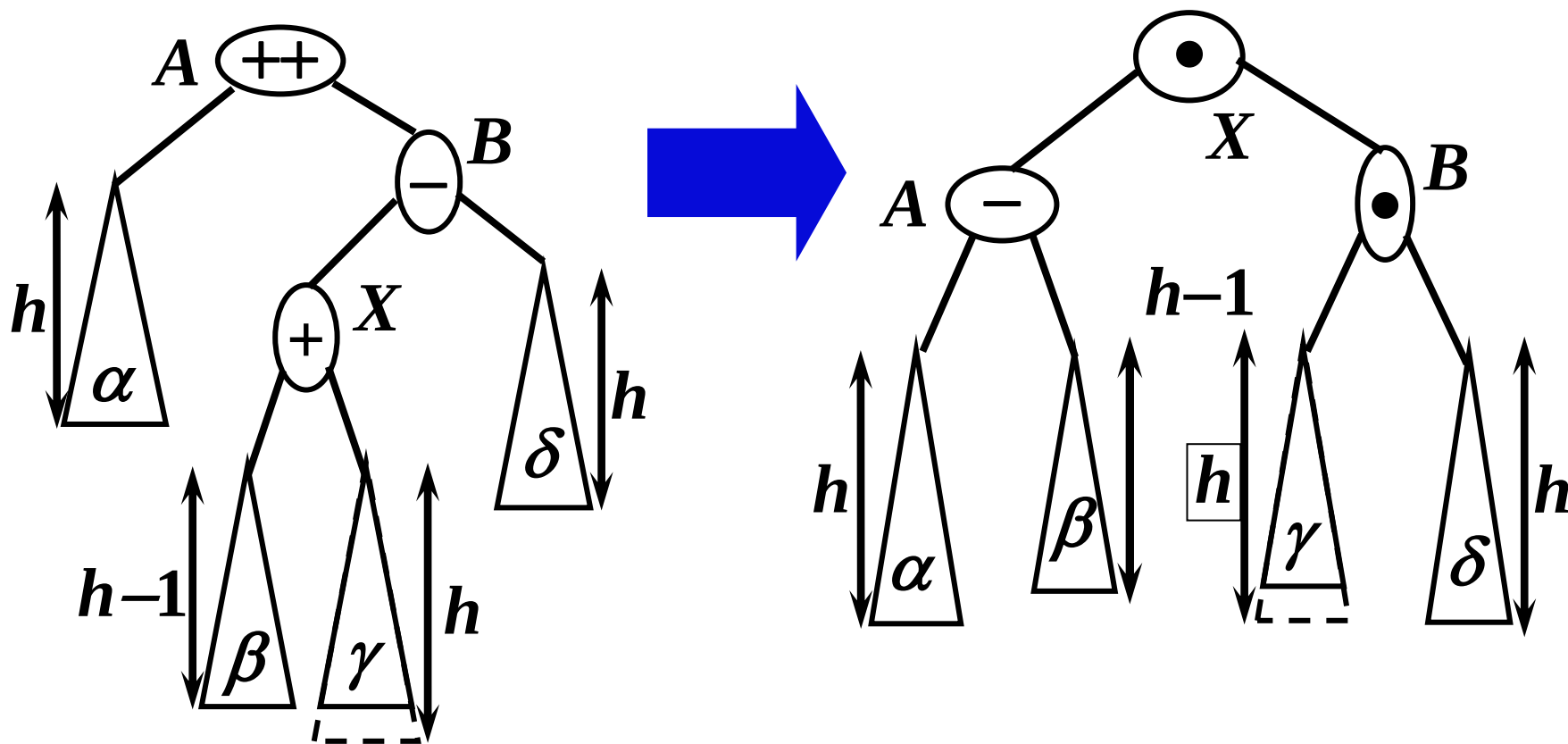
首先, 以 X 为轴将 B 从 X 的右上侧**右**转到 X 的右下侧, 记为 (X, B) , 从而 A 的右儿子是 X , X 的右儿子是 B , 原 X 的右子树 γ 变成了新 B 的左子树;



然后，以 X 为轴心，把 A 从 X 的左上方左转到 X 的左下侧，记为 (A, X) ，使 X 的左儿子是 A ，右儿子是 B ，原 X 的左子树 β 变成了 A 的右子树。

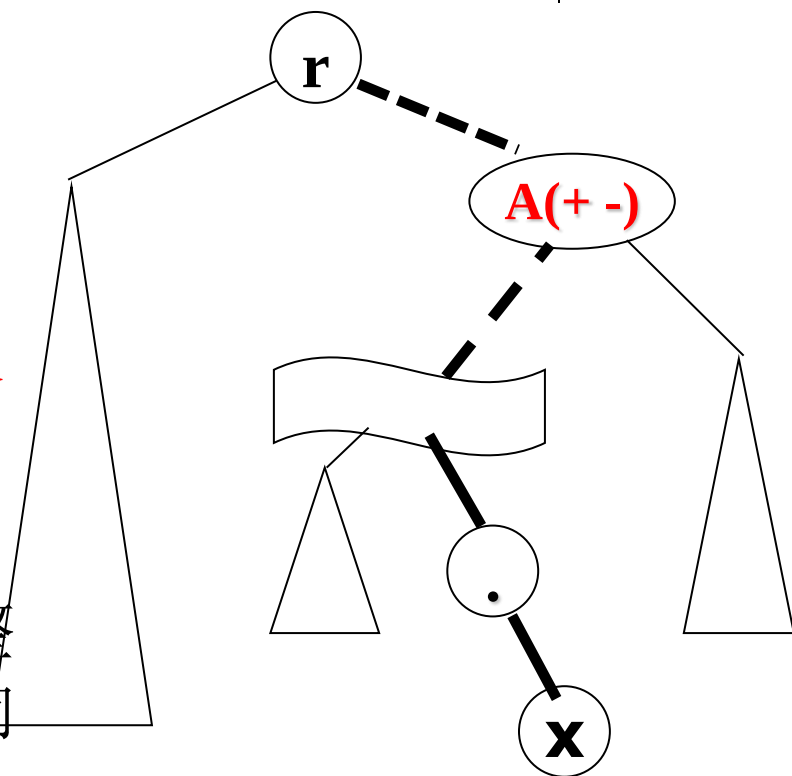


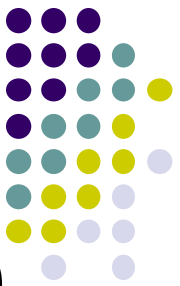
情况2: “双重转动”



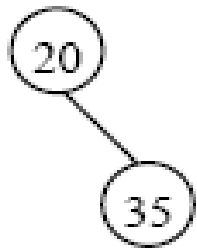
插入操作

1. 插入结点（同**BST**）
 2. 若树失去平衡，则调整。
 - ✓ 调整失去平衡的最小子树。记录从根结点r到结点x的路径上的最后一个不平衡点A，平衡以A为根的子树形即可。调整后，高度与调整前相同。其它结点不受影响
- 只需一次调整，插入算法的效率为 $O(\log N)$

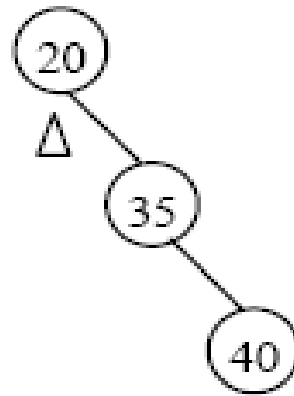




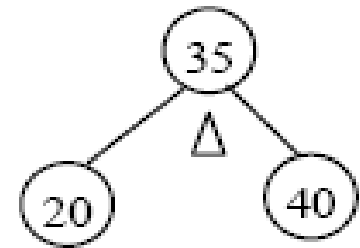
- 例：构造包含关键词{20, 35, 40, 15, 30, 25} 高度平衡树。



(a)

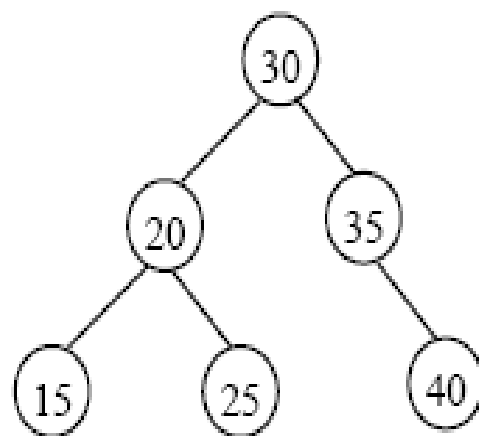
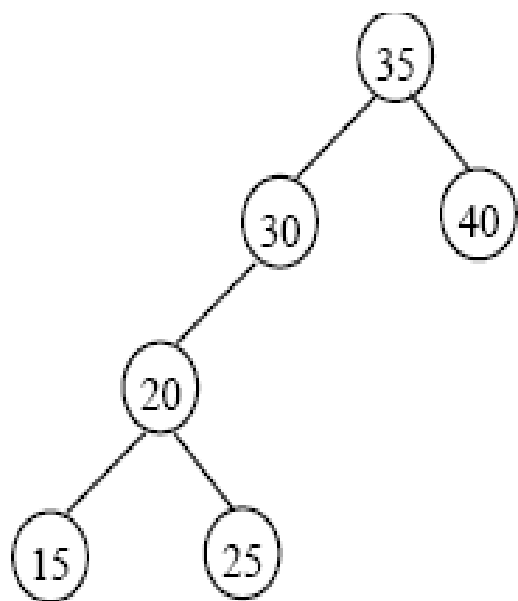
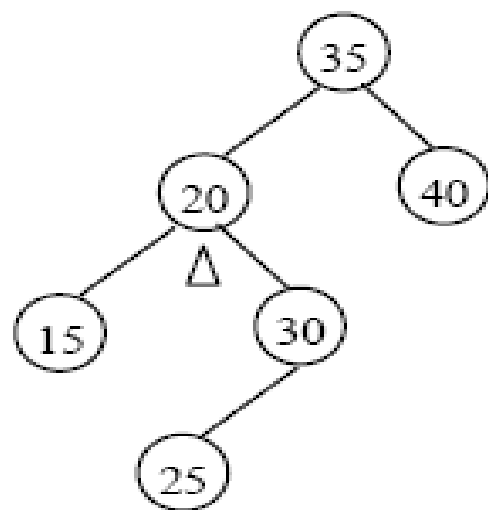
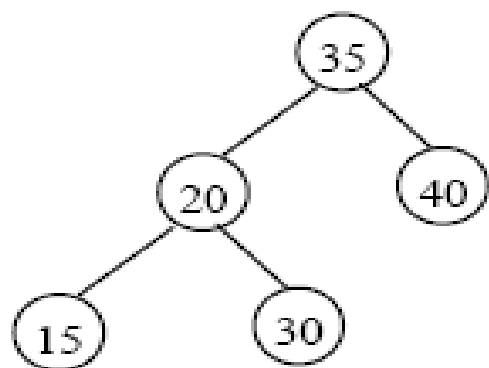


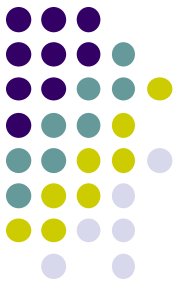
(b)



(c)

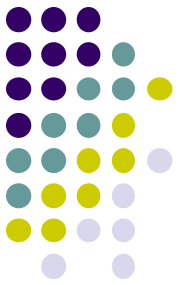
o





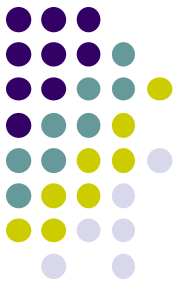
删除操作

1. 删除结点（同**BST**）
 2. 若树失去平衡，调整
- 只有被删除结点父亲的平衡性可能遭到破坏。但调整要从被删除结点到根结点，逐步向上调整。
 - 设从根到被删除结点的路径上的所有点依次入栈 **S**，**current** 指向被删除结点，**parent** 是 **current** 的父结点，



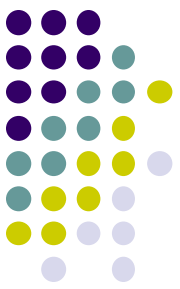
删除后调整规则

1. 如果 **parent->B==0**, 且 **current == parent->left**, 则 **parent->B = +1**, 否则 **B(parent) = -1**. 调整结束.
2. 如果 **parent->B==+1** 且 **current == parent->right**,
或 **parent->B==-1** 且 **current == parent->left**, 则 **parent->B=0**, 且 **parent** 的高度减1.
此时令 **current = parent**, **parent ← S**, 然后继续.



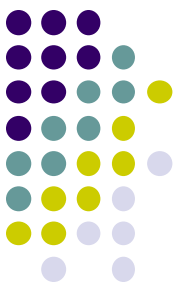
3.1 如果 $\text{parent} \rightarrow B == +1$ 且 $\text{current} == \text{parent} \rightarrow \text{left}$, 则此时 parent 的平衡系数等于 $+2$, 因此根据 $\text{parent} \rightarrow \text{right}$ 的平衡系数来进行重新平衡操作, 若调整后树高度不变, 则算法终止; 若调整后树高度减1, 则令 $\text{current} = \text{parent}$, $\text{parent} \leftarrow S$, 然后继续.

3.2 如果 $\text{parent} \rightarrow B == -1$ 且 $\text{current} == \text{parent} \rightarrow \text{right}$, 则此时所发生的情况正好是**3.1**所示情况的对称情形.



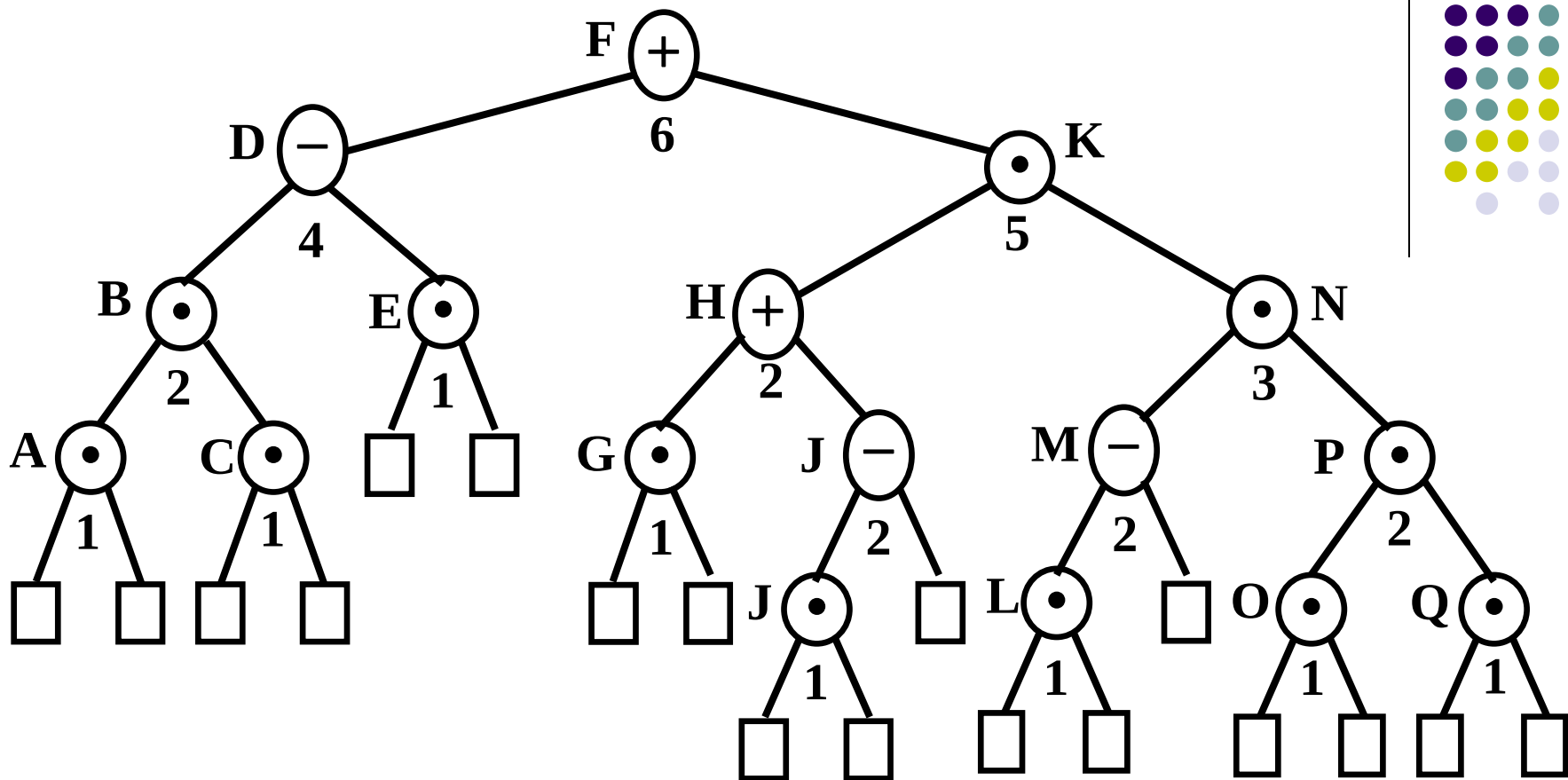
删除分析

- 删除后调整操作最多需 $O(\log_2 N)$ 次，每次 $O(1)$ ；删除操作的时间复杂度为 $O(\log_2 N)$ 。
- 删除和插入之间的重要差别是：删除可能需要多达平均 $\log_2 N$ 次调整，而插入所需的调整不多于1次。

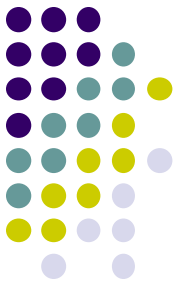


平衡树求第k小数

- 用平衡树来实现线性表，既可快速地插入（克服顺序存储的困难），又可对表实施随机存取（克服链接存储的困难）。
- 每一个结点P增加一个被称之为RANK的字段
 - ✓ $RANK(P) = P$ 的左子树形的(内)结点数 + 1
 - ✓ RANK域之值确定了结点的相对位置



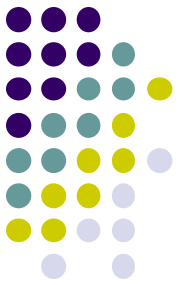
可方便地查找在中根次序下表的第 k^* 个结点. 假定待查结点的关键词为 k^* . 若 $k^* < 6$, 则向左去查找; 若 $k^* > 6$, 则往右去检索, 比如 $k^* = 8$, 则 **NODE (H)** 便是所求, 因 $RANK(H) = k^* - 6 = 2$. 又如, **NODE (E)** 在中根次序下的位置是 5, 它等于 $RANK(D) + 1$; **NODE (K)** 的中序位置是 11, 它等于 $RANK(F) + RANK(K) = 6 + 5$; 等等.



找第k小元素

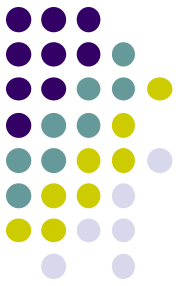
算法kth (t, k)

```
p=t;
while( p!=0 )
    if( k == tr[p].rank ) return p;
    else if(k<tr[p].rank) p=tr[p].l;
    else k-=tr[p].rank ,p=tr[p].r;
}
return p;
```



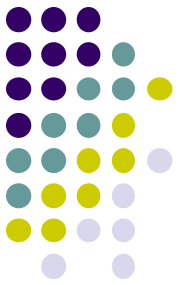
平衡树的优缺点

- 优点: **AVL**引入平衡因子, 很好的限制了树的平均高度为 $O(\log N)$, 导致插入、删除、查找的最坏时间复杂度均为 $O(\log n)$
- 缺点: **AVL**插入和删除的实现略难;
- 替代方案
 - ✓ **Splay**: 均摊时间复杂度为 $M \log N$
 - ✓ **Treap**: 引入随机数建立**BST**
 - ✓ 红黑树: 一种平衡树; 动态集合操作最坏 $O(\log n)$
 - ✓ SBT.....



平衡树参考实现

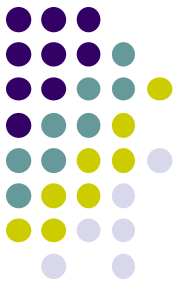
```
struct AVLNode
{
    int key;
    int height;
    AVLNode *left, *right;
    AVLNode(int K){ key=K; height=0;
left=right=NULL; }
};
```

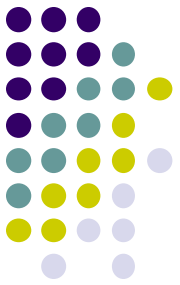
```
int getH(AVLNode *t){ return t==NULL ? -1 : t-  
>height; }
```

```
int max(int a, int b){ return (a>b)? a:b; }
```

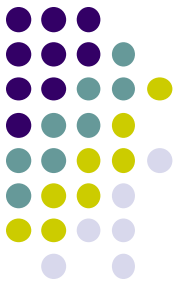
```
void updateH(AVLNode *t)  
{  
    t->height = max(getH(t->left),getH(t->right))+1;  
}
```



```
void LL(AVLNode* &A)  
{  
    AVLNode *B = A->left;  
    A->left = B->right;  
    B->right = A;  
    updateH(A);  
    updateH(B);  
    A = B;  
}
```



```
void RR(AVLNode* &A)  
{  
    AVLNode *B = A->right;  
    A->right = B->left;  
    B->left = A;  
    updateH(A);  
    updateH(B);  
    A = B;  
}
```



```
void LR(AVLNode* &A)
```

```
{
```

```
    RR(A->left);
```

```
    LL(A);
```

```
}
```

```
void RL(AVLNode* &A)
```

```
{
```

```
    LL(A->right);
```

```
    RR(A);
```

```
}
```

```
void Balance(AVLNode* &t) {
```

```
    if(getH(t->left) - getH(t->right)==2){
```

```
        if(getH(t->left->left) >= getH(t->left->right))
```

```
            LL(t);
```

```
        else
```

```
            LR(t);
```

```
    }
```

```
    else if(getH(t->right) - getH(t->left)==2){
```

```
        if(getH(t->right->right) >= getH(t->right->left))
```

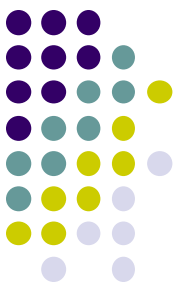
```
            RR(t);
```

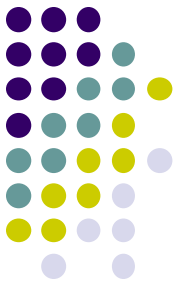
```
        else
```

```
            RL(t);
```

```
    }
```

```
    updateH(t);
```





```
void Insert(AVLNode* &root, int K)  
{  
    if(root==NULL) root=new AVLNode(K);  
    else if(K < root->key)  
        Insert(root->left, K);  
    else if(K > root->key)  
        Insert(root->right, K);  
  
    Balance(root);  
}
```

```
void Remove(AVLNode*& t, int k) {  
    if (t == NULL) return;  
    else if (k < t->key) Remove(t->left, k);  
    else if (k > t->key) Remove(t->right, k);  
    else {  
        if (t->left != NULL && t->right != NULL) {  
            AVLNode* s = t->right;  
            while (s->left != NULL) s = s->left;  
            t->key = s->key;  
            Remove(t->right, t->key );  
        }else {  
            AVLNode* tmp = t;  
            t = t->left ? t->left : t->right;  
            delete tmp;  
        }  
    }  
    if (t) Balance(t);  
}
```

